

Virtual Gold x CMU (Hybrid Architecture)

Capstone Midterm Presentation | October 5, 2026

CMU Capstone Team

Zhexuan Ye, Anmol Sahu, Yudi Guo, Karina Yang, Mark Chu

Heinz College of Information Systems and Public Policy

Carnegie Mellon University

Five team members each own one part of the service

1

Zhexuan Ye

Service engineering

2

Anmol Sahu

Model work

3

Yudi Guo

Privacy research

4

Karina Yang

Evaluation and test data

5

Mark Chu

Integration and coordination

Faculty advisor: Stacy Rosenberg

Client: Virtual Gold Inc.

One email shows the decisions the service must make

Before: the executive's email

Hi Sam,
Please call me on 412-555-0186
before the Project Falcon review.
I'm in Pittsburgh this week. For
help, use our support line,
1-800-555-0100.

Local
service

After: what the cloud assistant sees

Hi [PERSON_1],
Please call me on [PHONE_1]
before the [PROJECT_1] review.
I'm in Pittsburgh this week. For
help, use our support line,
1-800-555-0100.

High tier: redacted

Medium tier: redacted

Low or public: kept

Synthetic example. Placeholders such as [PHONE_1] can be restored locally; the mapping never leaves the service.

Figure 1. Synthetic email before and after redaction. Source: Project team.

Project scope: redact private data locally before it reaches the cloud

1

Problem

Executive emails contain private data.

A cloud assistant should see only what it needs for the task.

2

Deliverables

Local redaction and restore service

Sensitivity tiers and decision rules

100-email evaluation benchmark

Monitoring dashboard

3

Constraints

Run on project hardware with an open-source model

Allow no outbound network access

Work automatically on every request

Finish by the final presentation on November 30

Agenda

1

Sensitive Data Classification

Yudi Guo

2

Architecture Design

Zhexuan Ye, Anmol Sahu

3

Service Engineering Plan

Zhexuan Ye

4

Evaluation Dataset

Karina Yang

5

Summary and Next Steps

Mark Chu

Sensitive Data Classification

Yudi Guo

Six client requirements and how the proposal meets each

Requirement	How the proposal meets it
Strip private data before the cloud	Redact with reversible placeholders and restore the reply
Run locally on an open-source model	Run on own hardware with no outbound network access
Work automatically on every request	Decide in code on every request, with no per-item review
Judge context, not just patterns	Read context, such as “my cell” vs. “our support line”
Output a confidence score	Calibrate the model score on labeled emails
Measure on 100 annotated emails	Track leak rate per tier across versions

Stated main challenge: determining what counts as private.

Table 1. Client requirements and how the proposal meets them. Note. Requirements from the Virtual Gold project brief.

Four prior works shaped our design choices

Source	Key finding	How it shaped our design
McCallister et al. (2010), NIST SP 800-122	Harm from exposed PII differs by type and context	Three sensitivity tiers with different thresholds
Microsoft (n.d.), Presidio	Open-source rules and entity recognition find common PII	One of three candidate detectors
Zhou et al. (2025), Rescriber	A local Llama3-8B found 63% of personal data (GPT-4o 88%, Presidio 68%)	Combine three detectors and redact by default
Xiong et al. (2024)	Language models are overconfident in their own answers	Calibrate raw scores on labeled emails

Table 2. Related research and how it informed the design.

*The category sets how severe a leak would be.
The context sets how confident we are.*

When confidence is insufficient, the item is redacted by default and replaced with a reversible placeholder, so the assistant can still do its job.

The model judges – code decides

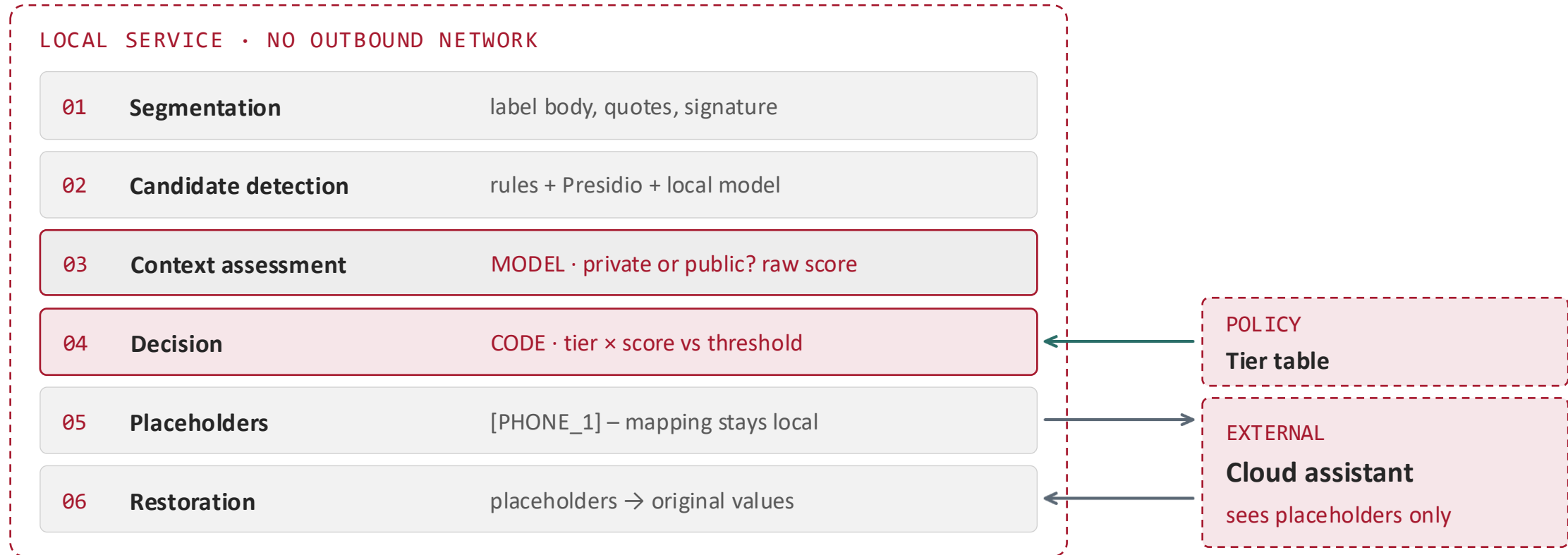


Figure 2. Decision steps inside the local service. Source: Project team.

More harm, less proof required

Tier	Categories	Redaction rule
High	IDs, passwords and keys, bank and card numbers, health, business-confidential	Redact at score ≥ 0.2
Medium	Names of private individuals, personal phone, personal email, home address	Redact at score ≥ 0.5
Low	City, dates, organisation names, age, gender	Keep alone – redact if combined

Table 3. Sensitivity tiers and redaction rules. Note. Tiers follow NIST SP 800-122 (McCallister et al., 2010). Thresholds are starting values.

Two phone numbers, two outcomes

INCOMING EMAIL

Hi Tom,
 Before Thursday's board call: my **oncologist** moved my appointment, so I'll dial in from home. Reach me on my cell, **412-555-0182**.
 On the vendor outage, Acme's support line is **1-800-555-0199**.
 — Dana

Item	Context	Score	Decision
Dana, Tom	private individuals	0.90	[PERSON_n]
oncologist	"my appointment"	0.95	[HEALTH_1]
412-555-0182	"my cell"	0.95	[PHONE_1]
1-800-555-0199	toll-free, "support line"	0.15	Keep
Acme, Thursday	Low tier, alone	n/a	Keep

Table 4. Items in the example email and the resulting decisions. Note. Synthetic example, scores are illustrative.

What the cloud assistant receives

SENT TO THE CLOUD

Hi **[PERSON_2]**,
Before Thursday's board call: my **[HEALTH_1]** moved my appointment, so I'll dial in from home. Reach me on my cell, **[PHONE_1]**.
On the vendor outage, Acme's support line is 1-800-555-0199.— **[PERSON_1]**

Still possible

Draft the reply to **[PERSON_1]**. Summarise the Acme outage.

Never leaves the machine

The diagnosis, the personal number, both names.

On the way back

Placeholders are restored locally before the executive reads the draft.

Calibrating the model's score

Language models tend to be overconfident about their own answers (Xiong et al., 2024). The raw score is therefore corrected before any threshold is applied.

1. Run the model on labelled emails
2. Group items by raw score
3. Measure how many are actually private
4. Use that rate as the calibrated score

Raw score	Actually private	Calibrated
0.9 – 1.0	140 of 200	0.70
0.7 – 0.9	50 of 100	0.50
0.5 – 0.7	15 of 50	0.30
0.0 – 0.5	15 of 150	0.10

Table 5. Example calibration of raw model scores. Note. Illustrative values: a raw 0.95 would be reported as 0.70.

One third of personal data missed

0.63

Recall of a local Llama3-8B in the Rescriber study (Zhou et al., 2025): about one in three personal items missed. GPT-4o reached 0.88 and Presidio 0.68.

Rescriber's users reviewed every message. Here, a miss goes straight to the cloud. After measuring our own baseline on email, we add one measure at a time:

- Combine all three detectors
- Lower tier thresholds
- Short email examples in the prompt
- ...

Leak rate per tier is the primary evaluation metric

DATA

100 emails

Every private item labelled by category, and private or public in context

PRIMARY METRIC

Leak rate

Share of private items that reach the cloud, per tier

ALSO TRACKED

Utility

Over-redaction, calibration, seconds per email

Ablation: each step is added to a Rescriber-style baseline, tuned and tested on separate subsets

Baseline

+ detector union

+ name propagation

+ repeated runs

+ tier thresholds

The service combines existing components with two new contributions

Component	Effort
Rules and Presidio	Low
Local model via Ollama, using Rescriber's open-source prompt	Low to medium
Merging the three detectors' outputs	About 50–100 lines

OUR CONTRIBUTIONS

1. Running without a person reviewing each item. Rescriber, the closest prior system, asks the user to confirm every redaction.
2. A labelled email benchmark and an ablation showing what each part adds.

Table 6. Existing components and estimated integration effort. Note. Speed depends on hardware. Zhou et al. (2025) report a 3.4 s mean for Llama3-8B on an M2 laptop.

Architecture Design

Zhexuan Ye, Anmol Sahu

Three principles shape every design decision

1

Local by default

All detection and judgment run on project hardware, with no outbound network access.

2

Model proposes, code decides

The model finds and scores spans – tier rules in code make every redaction decision.

3

Fail safe

When a component fails or an item cannot be classified, the service redacts.

One local model sits inside a pipeline of code and data stores

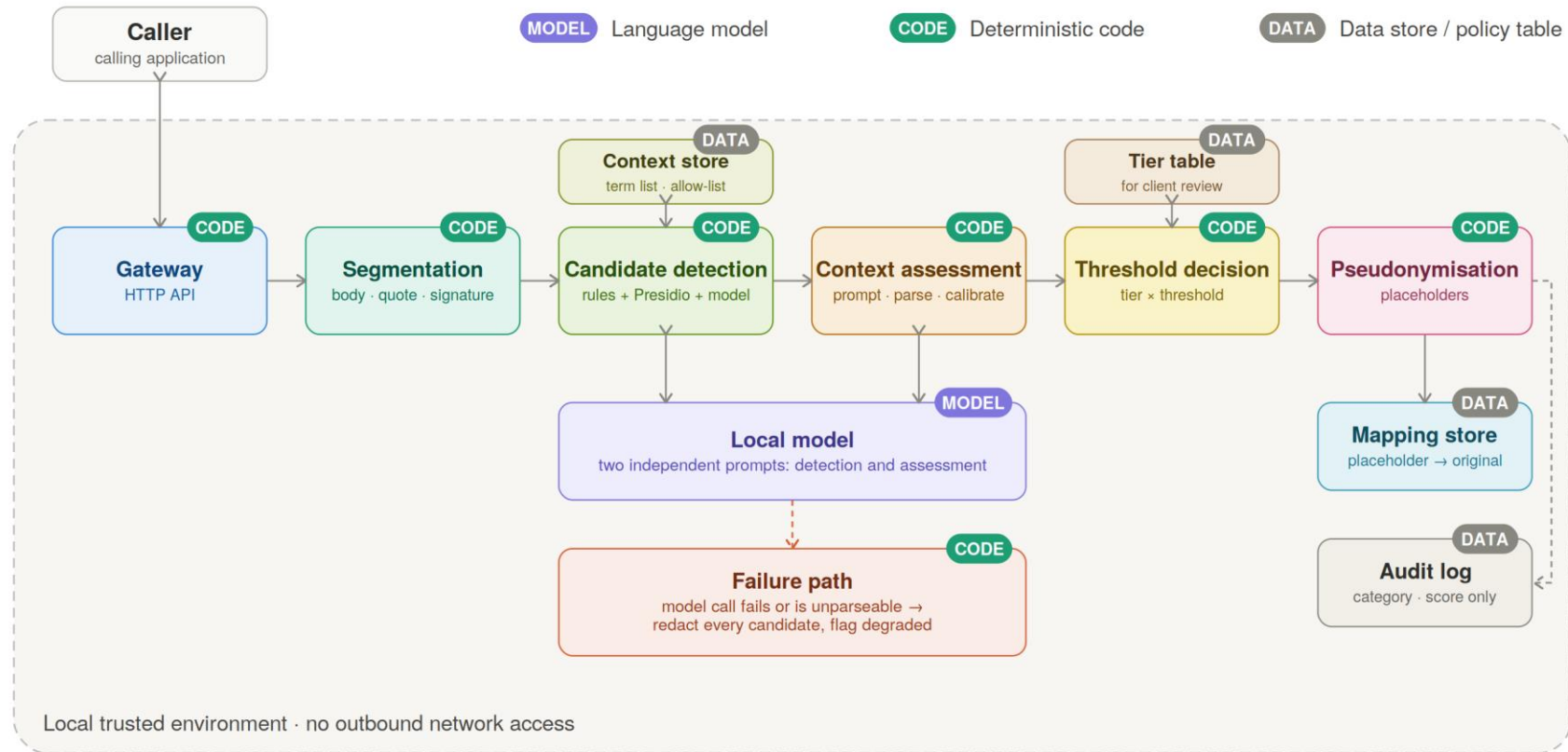
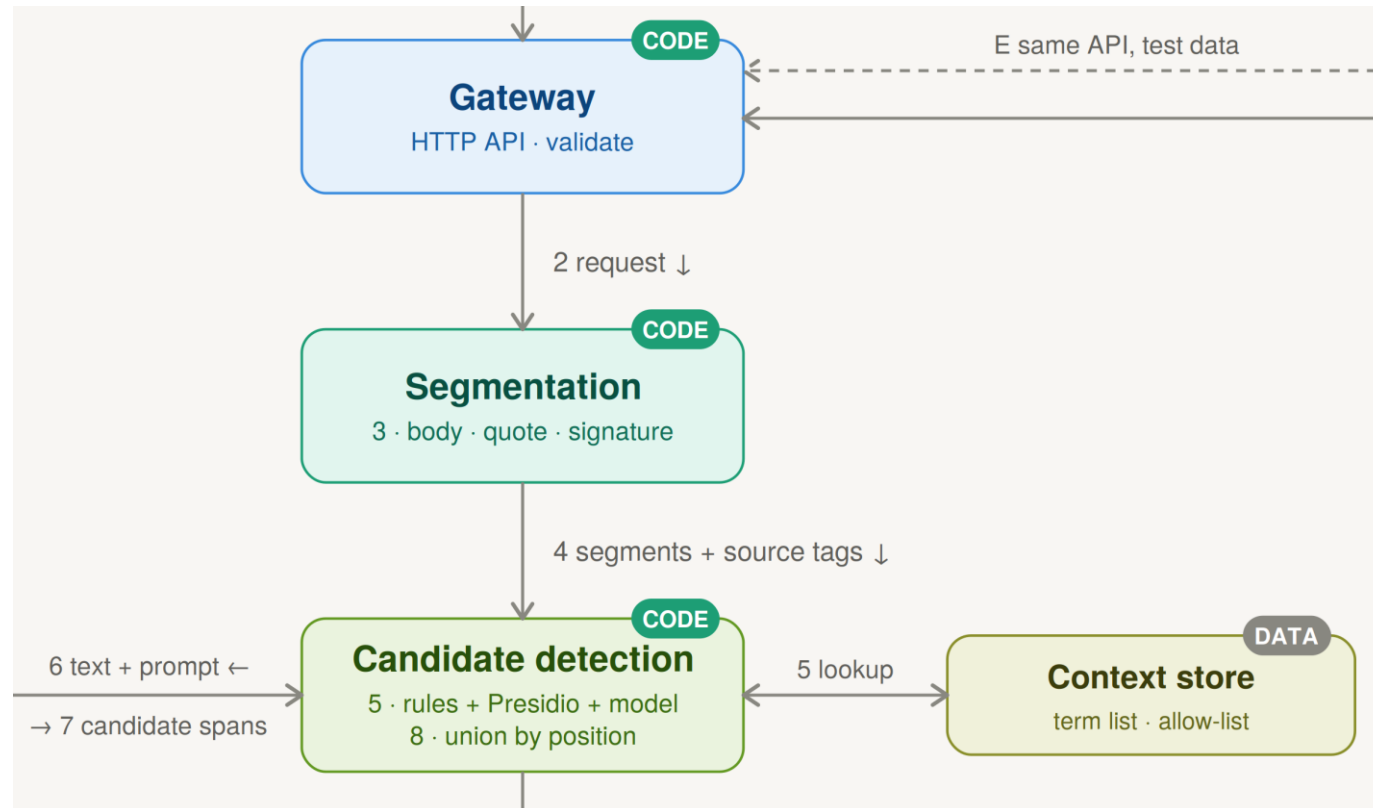


Figure 3. Component map of the local redaction service. Source: Project team.

Segmentation and three-way detection turn raw text into candidate spans



3 Split into body, quote and signature

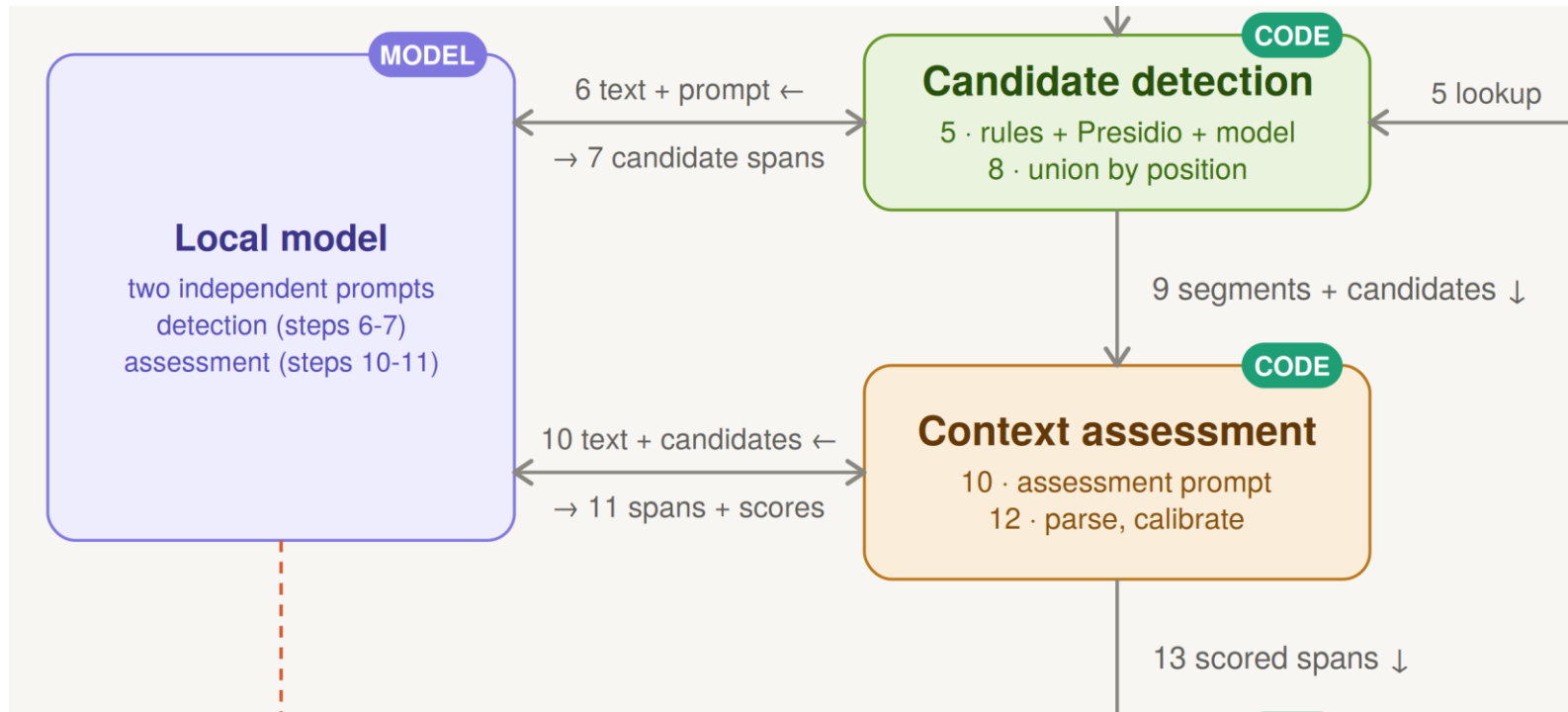
4 Pass segments with source tags

5 Run rules and Presidio – check the context store

6–8 Add model candidates – union all three sources

Figure 4. Input side: gateway, segmentation and candidate detection (rules, Presidio and model).

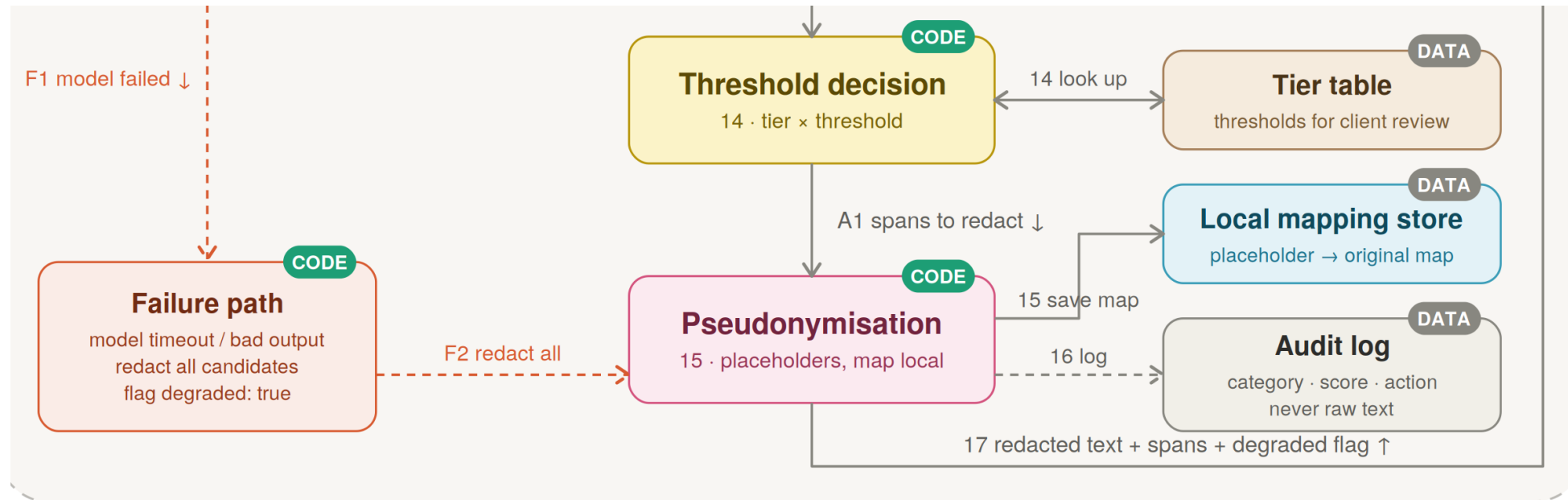
One local model serves detection and assessment with independent prompts



Detection prompt: steps 6–7. Assessment prompt: steps 10–11.

Figure 5. One local model, called with two independent prompts. Source: Project team.

Code applies the tiers, then replaces spans with placeholders



14 Tier table sets the threshold for each span

15 Placeholders replace spans – the map stays local

16 Audit log records category and score, never raw text

Figure 6. Decision, pseudonymisation, mapping store, audit log and failure path. Source: Project team.

Steps 1–5: the request is split and rule-based candidates are found

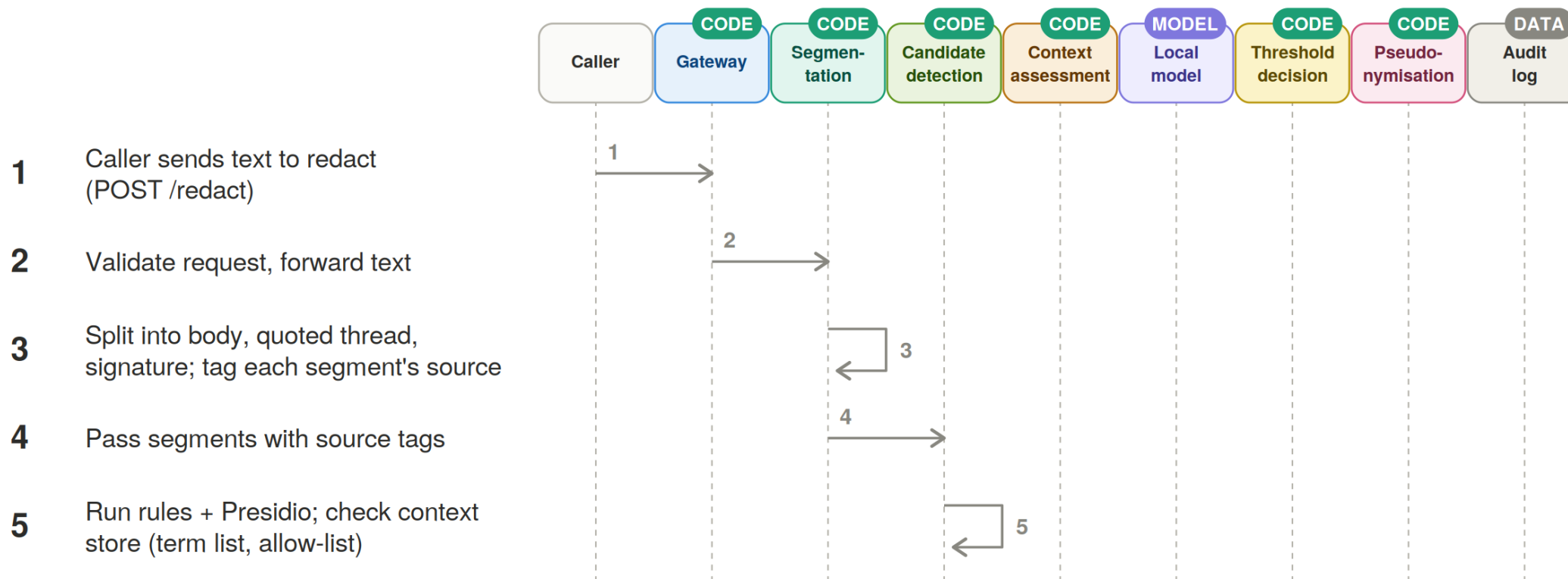


Figure 7. Request workflow, steps 1–5. Source: Project team.

Steps 6–11: the model is called twice, with two independent prompts

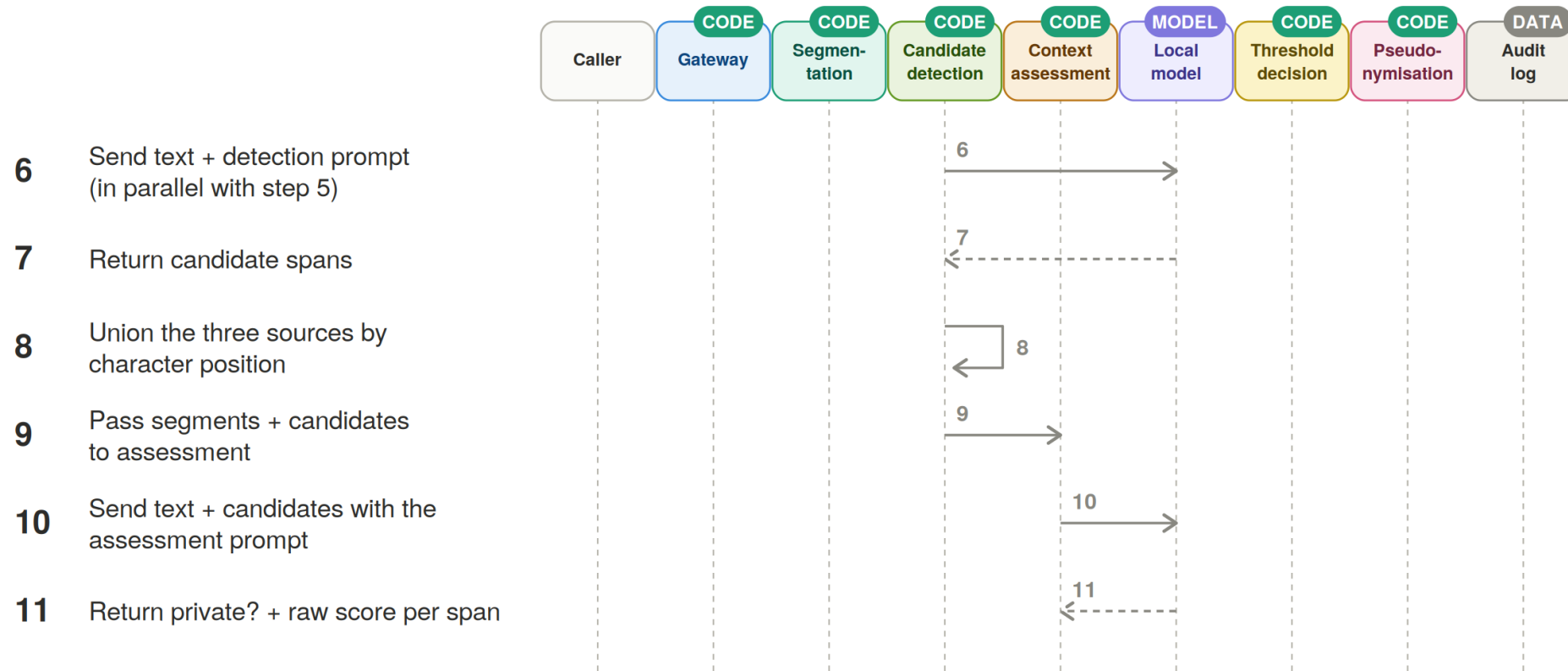


Figure 8. Request workflow, steps 6–11. Source: Project team.

Steps 12–14: code turns scores into decisions – only qualifying spans are replaced

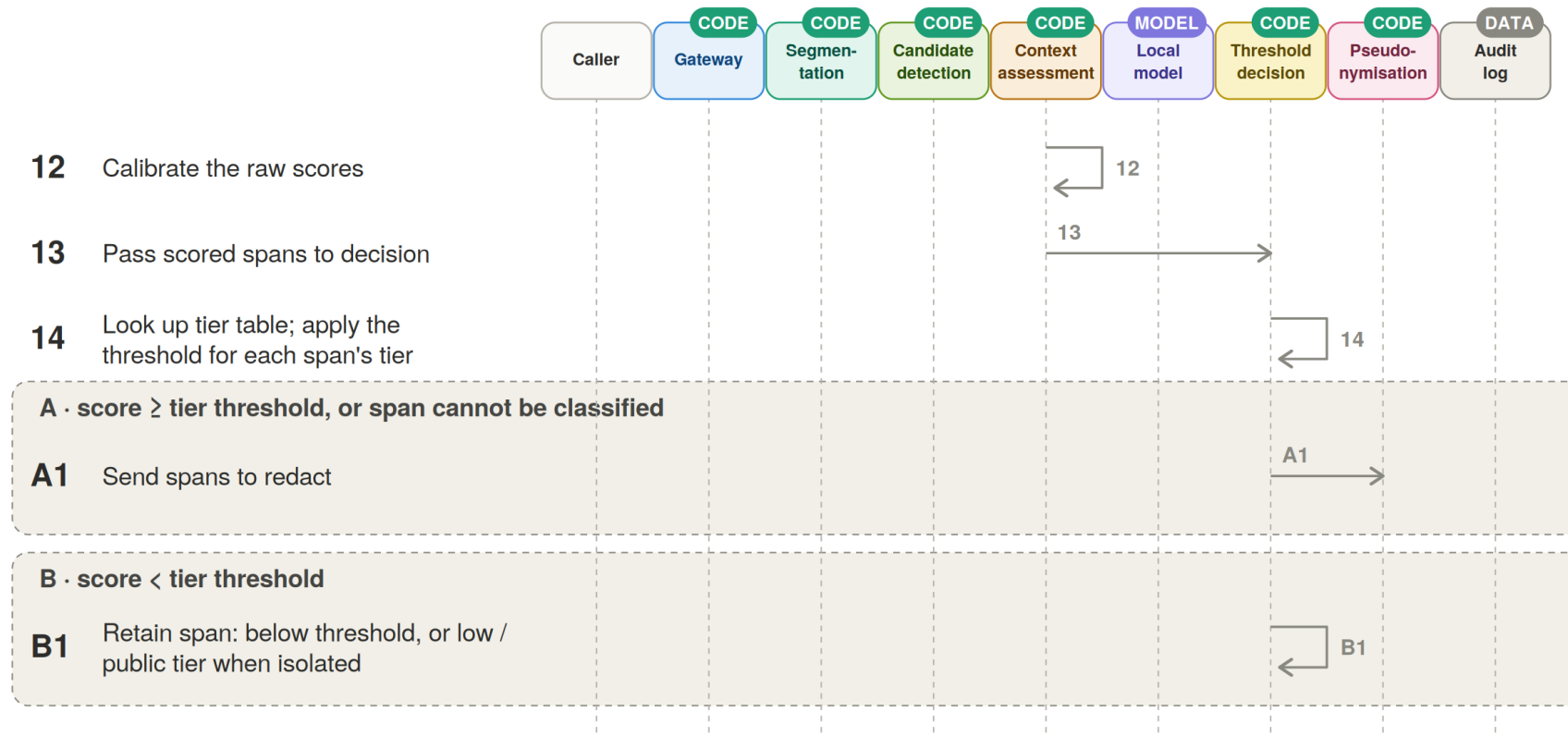


Figure 9. Request workflow, steps 12–14 and decision outcomes A and B. Both branches continue to step 15.
Source: Project team.

Steps 15–18: redacted text returns and the decision is logged

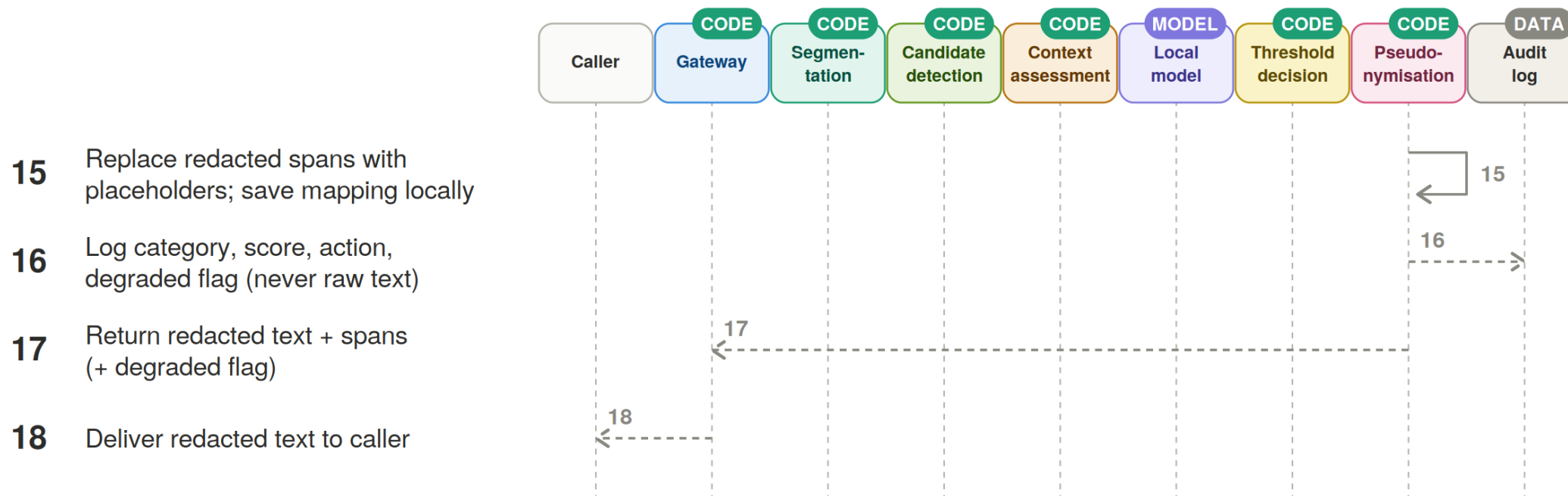


Figure 10. Request workflow, steps 15–18. Source: Project team.

Restore runs later as a separate, deliberate call

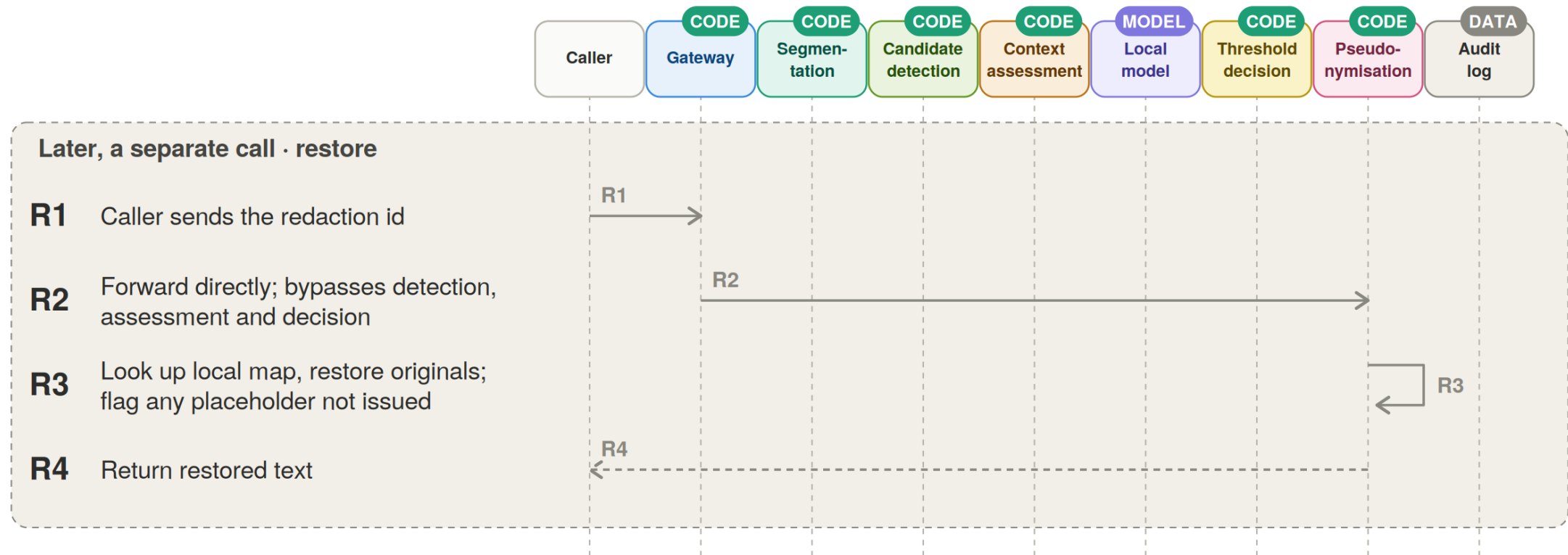
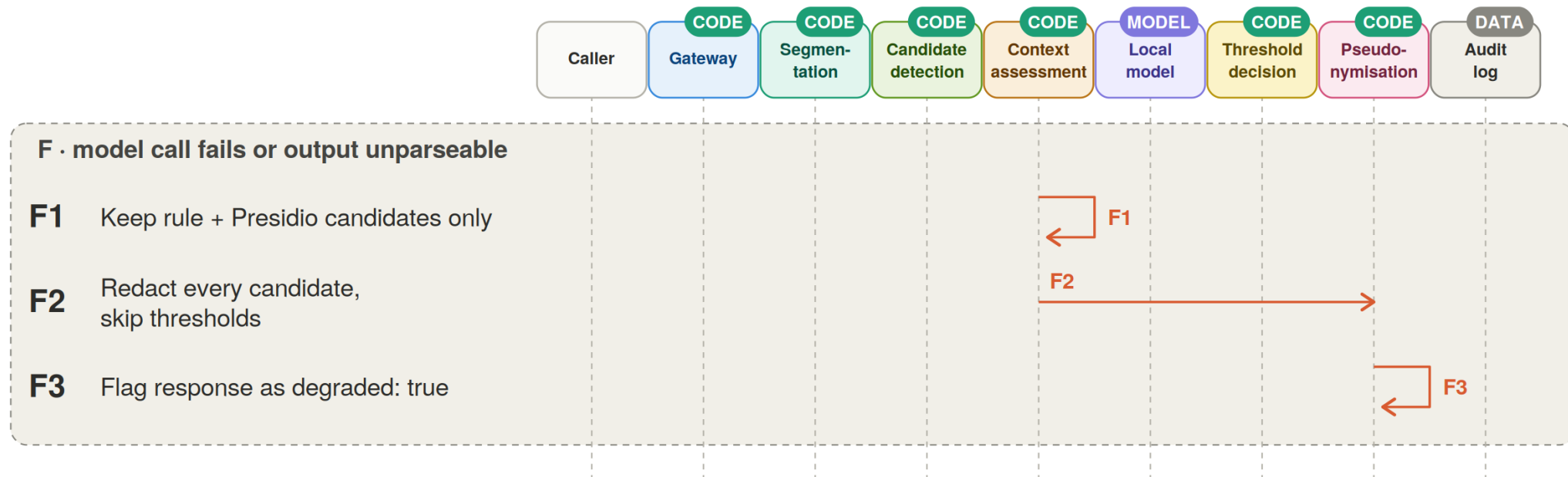


Figure 11. Restore request, handled separately from redaction. Source: Project team.

Any failure redacts every candidate and flags the response



Also flagged: any placeholder in a reply that the service did not issue.

Figure 12. Failure path when the model is unavailable or its output cannot be parsed. Source: Project team.

Service Engineering Plan

Zhexuan Ye

Service engineering owns the gateway and the wiring of the whole service

We build

- Gateway: HTTP API and validation
- Pipeline wiring and failure path
- Request records: restore and delete
- Metrics and logs, never raw text
- Dashboard: Prometheus and Grafana

A live dashboard will show the health of the service

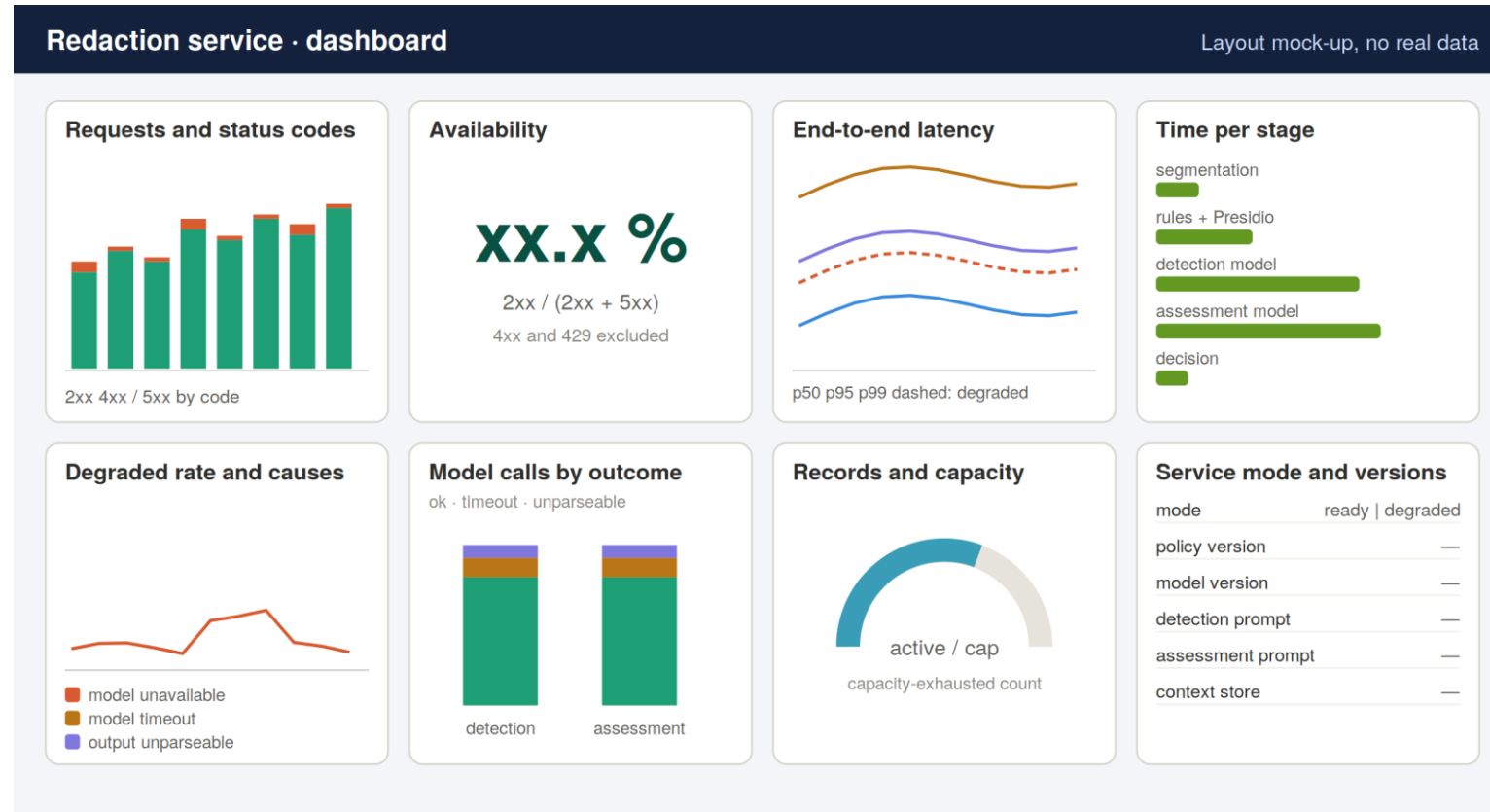


Figure 13. Planned dashboard layout (mock-up, no real data).

Evaluation Dataset

Karina Yang

The dataset holds 100 fictional emails with 999 labeled items

100

realistic executive emails, all fictional

15

business areas, from finance to personal

9

industries and 4 exec roles

999

labeled sensitive items, 18 types

TEST CASES BUILT IN

Aarvin's context cases

Personal cell next to a public support line

10 clean emails

Nothing sensitive, to measure over-masking

Forwarded threads

Names quoted inside reply chains

5 attack emails

Hidden prompt injection, spelled-out numbers

56 business secrets

Sensitive sentences with no single name

Each email is labeled by business context and by sensitive item

EMAIL E001 • FINANCE • CFO

From: **Megan Ostrowski** PERSON

Victor PERSON, the warranty reserve needs to go up by **\$1.84M** AMOUNT because of the pump returns.

Office: **(617) 555-0142** PHONE

Task: "Summarize this and tell me what I need to approve."

Secrets with no single name, like "we're closing the Fresno site", get a **FACT** label.

Per email	Example
Business area	Finance & Accounting
Industry	Medical devices
Exec role	CFO
Task	Summarize, flag approvals

Per label	Meaning
Type	One of 18: person, phone, amount, codename...
Position	Exact start and end character
must_mask	true = leak if sent raw – false = public, fine either way

Table 7 (top). Fields recorded for each email. Table 8 (bottom). Fields recorded for each labeled item. Source: Project team, fictional data.

One JSON line per email links the dataset to model output

DATASET RECORD • ONE LINE PER EMAIL

```
{
  "id": "E001",
  "domain": "Finance & Accounting",
  "task": "Summarize this and...",
  "text": "From: Megan Ostrowski...",
  "spans": [
    {"start": 6, "end": 21,
     "type": "PERSON",
     "must_mask": true}
  ]
}
```

Also records **industry**, **exec** role and **tags**. **spans** is the answer key: every sensitive item and its exact position.

MODEL OUTPUT • ONE LINE PER EMAIL

```
{
  "id": "E001",
  "spans": [
    {"start": 6, "end": 21,
     "type": "PERSON",
     "score": 0.93}
  ]
}
```

The model reads only **text** and **task**, never the answer key. It returns what it masked: **start** and **end** are required, **type** and confidence **score** are optional.

Summary and Next Steps

Summary: one local service, four workstreams

1

Sensitive data classification

Category sets severity,
context sets confidence

Calibrated confidence
scores

Main risk: missed
personal data

2

Architecture design

Everything runs locally

The model proposes,
code decides

Any failure redacts
every candidate

3

Service engineering

Gateway, pipeline, and
request records

Six milestones to
November 30

Live Prometheus and
Grafana dashboard

4

Evaluation dataset

100 fictional executive
emails

999 labeled items, 18
types

One shared format to
score every model

Six milestones lead from skeleton to final presentation

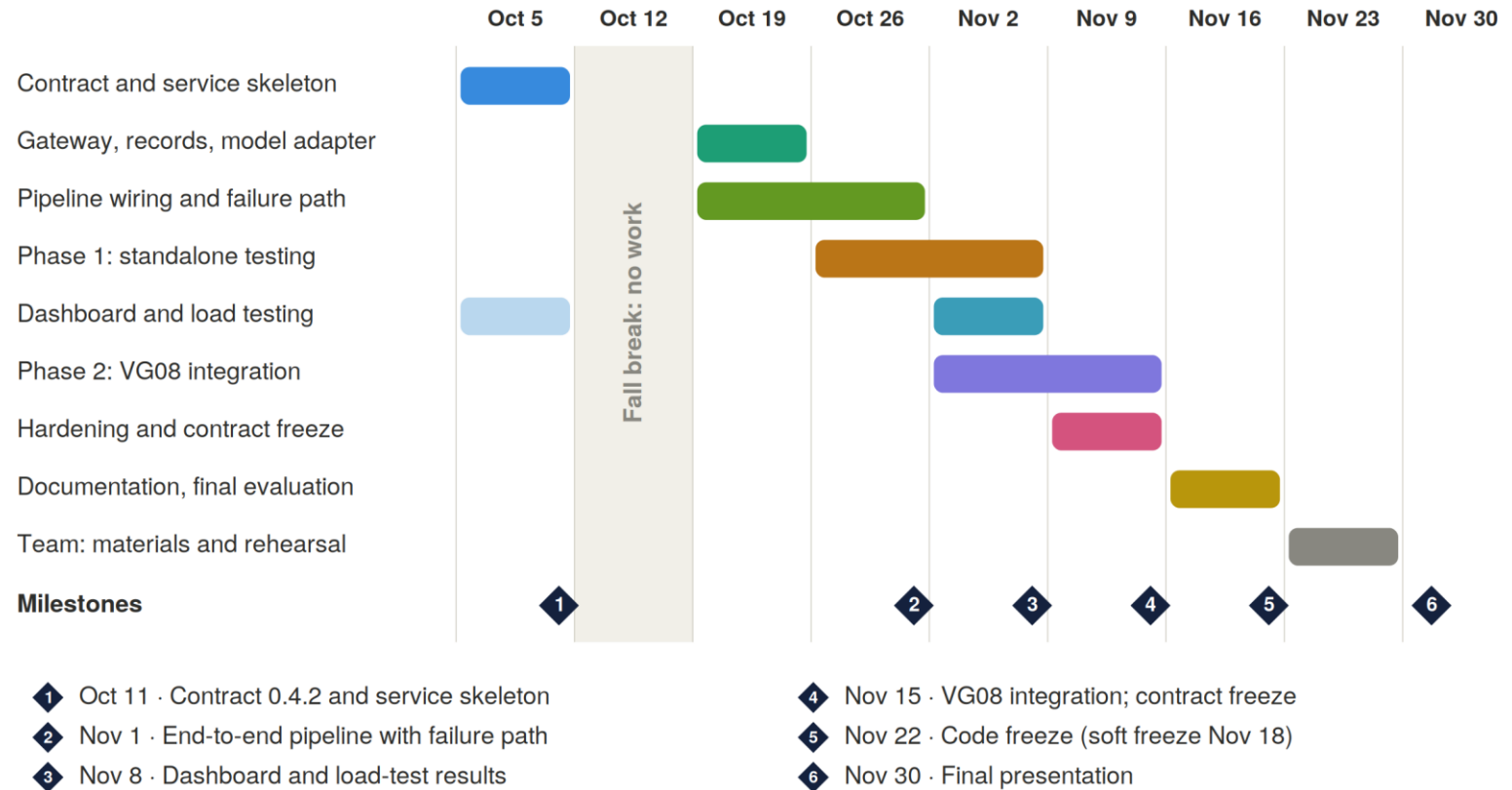


Figure 14. Service engineering schedule, October 5 to November 30. Source: Project team.

Next steps: two test phases, four inputs from the client

What we will do

- 1 Test the architecture with open-source models, cloud-hosted or local
- 2 Phase 1: test the redaction service on its own, from late October
- 3 Phase 2: integrate with the VG08 service, from November 2

What we need from you

- 1 Review the tier table and term list
- 2 Confirm whether the API request can label each text's source
- 3 Confirm GPU and sandbox access for the local model
- 4 Make VG08 available for integration

Questions?

Team contacts for follow-up

Zhexuan Ye	Service engineering	zhexuany@andrew.cmu.edu
Anmol Sahu	Model work	anmolsah@andrew.cmu.edu
Yudi Guo	Privacy research	yudig@andrew.cmu.edu
Karina Yang	Evaluation and test data	karinay@andrew.cmu.edu
Mark Chu	Integration and coordination	markchu@andrew.cmu.edu

References

- McCallister, E., Grance, T., & Scarfone, K. (2010). Guide to protecting the confidentiality of personally identifiable information (PII) (NIST Special Publication 800-122). National Institute of Standards and Technology.
<https://doi.org/10.6028/NIST.SP.800-122>
- Microsoft. (n.d.). Presidio: Data protection and de-identification SDK. <https://microsoft.github.io/presidio/>
- Xiong, M., Hu, Z., Lu, X., Li, Y., Fu, J., He, J., & Hooi, B. (2024). Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs. In Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024).
- Zhou, J., Xu, E., Wu, Y., & Li, T. (2025). Rescriber: Smaller-LLM-powered user-led data minimization for LLM-based chatbots. In Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems. ACM.